

One pass to find every "next bigger" number

Brute force checks every pair. A stack does it in $O(n)$.



For each number → the first **larger** number on its right

[5, 2, 4, 6, 1] → [6, 4, 6, -1, -1]

Next Largest Number to the Right

For each `nums[i]`, the first value to its right that is strictly larger

● Question

Given an integer array *nums*, return *res* where `res[i]` is the first number to the **right** that is larger than `nums[i]`.

If no larger number exists to the right, set `res[i] = -1`.

{ } Example

```
nums = [5, 2, 4, 6, 1]
```

```
res  = [6, 4, 6, -1, -1]
```

5 → next larger is 6

2 → 4 · 4 → 6

6 → nothing bigger → -1

1 → nothing to the right → -1

Flip the question

Brute force is $O(n^2)$ — scan right from every element

Brute force

For each number, walk right until you find a bigger one.

$O(n^2)$ time

For each of n numbers we may scan the whole rest of the array.

New perspective

Instead of "what's the next larger for this value?", ask: "**for which values on the left could THIS value be the answer?**"



6 is the next larger number of both **5** and **4**.
→ Scan **right** → **left**.

Candidates form a decreasing stack

Scan right→left. Drop any candidate $\leq$ the new value.

Candidates stay sorted:



↑ **top = smallest**

strictly decreasing

The rule

- New number arrives
- **Pop** every candidate $\leq$ it (they can't win)
- Add the new number on top

Add on top, remove from top = a Stack

A **monotonic decreasing stack** — the perfect structure for "next greater" problems.

Walkthrough: [5, 2, 4, 6, 1]

Right → left. Pop ≤ current · read top · push current

i	nums[i]	pop ≤ it	stack after	res[i]
4	1	—	[1]	-1
3	6	pop 1	[6]	-1
2	4	—	[6, 4]	6
1	2	—	[6, 4, 2]	4
0	5	pop 2, pop 4	[6, 5]	6

res = [6, 4, 6, -1, -1]

The Code

Each value is pushed once and popped at most once

```
def next_largest_number_to_the_right(nums):  
    res = [0] * len(nums)  
    stack = [] # candidates, decreasing  
  
    for i in range(len(nums) - 1, -1, -1): # right → left  
        while stack and stack[-1] <= nums[i]:  
            stack.pop() # drop values that can't win  
        res[i] = stack[-1] if stack else -1  
        stack.append(nums[i]) # current becomes a candidate  
  
    return res
```

Complexity + Takeaway

One pass, one stack

Each value

push
×1

pop
≤1

O(n) total

Complexity Analysis

Time Complexity

— each value pushed & popped once

O(n)

Space Complexity

— stack may hold all n values

O(n)

Takeaway: "Next greater / next smaller" → a **monotonic stack**. Scan one direction, pop what can't win, read the top. Same pattern powers Daily Temperatures and stock-span problems.